

Optimistic and Pessimistic Locking

Chapter 04 - JPA / Hibernate

Sub-Chapter 04-07

Lost Update Problem	Two users overwrite each other silently
Optimistic Locking	@Version detects conflict at commit time
OptimisticLockException	Catch, retry, and inform the user
Pessimistic Locking	SELECT FOR UPDATE locks the row early
LockModeType Options	OPTIMISTIC, PESSIMISTIC_READ, PESSIMISTIC_WRITE
Optimistic vs Pessimistic	When to use each strategy

1 The Lost Update Problem

Concurrency gone wrong - two writers, one winner, one loser

What is a Lost Update?

When two users load the same database record at the same time, edit it independently, and then both save, the second save quietly overwrites the first. The first user's changes are silently lost - no error, no warning. This is called the **lost update problem** and it is the most common data-integrity bug in multi-user web applications.

Step-by-step scenario

User A loads account balance: 1000.

User B loads account balance: 1000 (same row, same moment).

User A adds 500 and saves: balance = 1500.

User B subtracts 200 and saves: balance = 800.

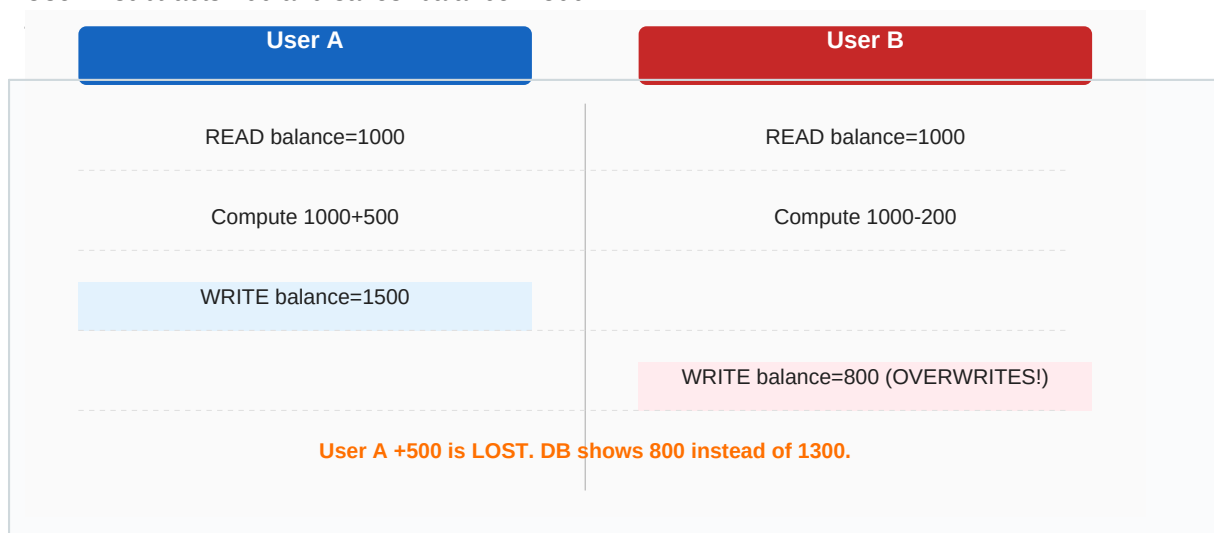


Figure 1 - Lost Update: both users read 1000; User B write silently discards User A change.

Java code showing the problem (no locking)

```
// NO locking -- dangerous with concurrent users
@Transactional
public void deposit(Long accountId, int amount) {
    Account acc = em.find(Account.class, accountId);
    // Another thread may have already changed balance here!
    acc.setBalance(acc.getBalance() + amount);
    // JPA flushes updated balance at commit -- may overwrite!
}
```

Key Rules

- Any multi-user app that edits shared rows is vulnerable to lost updates.
- The bug is silent - no exception is thrown; data is just wrong.
- You must choose either Optimistic or Pessimistic locking to prevent it.
- Read-only queries are never affected; only concurrent writes are a problem.

Q1. What is a lost update?

When two concurrent transactions both read the same row and then both write back their changes, the second write silently overwrites the first, erasing its changes.

Q2. Why does JPA not prevent lost updates automatically?

By default JPA uses READ COMMITTED isolation. It prevents dirty reads but does not detect that another transaction changed the row between your read and your write.

Q3. Is a lost update always visible to the user?

No. The application shows no error. The user who saved first simply finds their change gone the next time they reload the page.

Q4. What kind of data is most at risk?

Financial balances, inventory counts, vote tallies, and any numeric field that multiple users update concurrently.

Q5. What are the two standard solutions?

Optimistic locking (version column checked at commit) and Pessimistic locking (row locked at read time with SELECT FOR UPDATE).

2 Optimistic Locking

@Version - detect conflict at commit time, not at read time

How Optimistic Locking Works

Optimistic locking assumes conflicts are *rare*. Instead of blocking other users, it lets everyone read the row freely. It adds a **version column** to the table. When you commit, JPA checks: is the version in the database still the same version I read? If yes, the update proceeds and the version is incremented. If no, someone else already updated the row, so JPA throws an **OptimisticLockException** and rolls back your transaction.

Adding @Version to an Entity

```
@Entity
public class Account {

    @Id
    @GeneratedValue
    private Long id;

    private int balance;

    @Version          // JPA manages this column automatically
    private int version;

    // getters and setters ...
}
```

What SQL JPA generates

```
-- READ (normal select, no lock):
SELECT id, balance, version FROM account WHERE id = 1;

-- WRITE (includes version check in WHERE clause):
UPDATE account
  SET balance = 1500, version = 2
  WHERE id = 1;
```

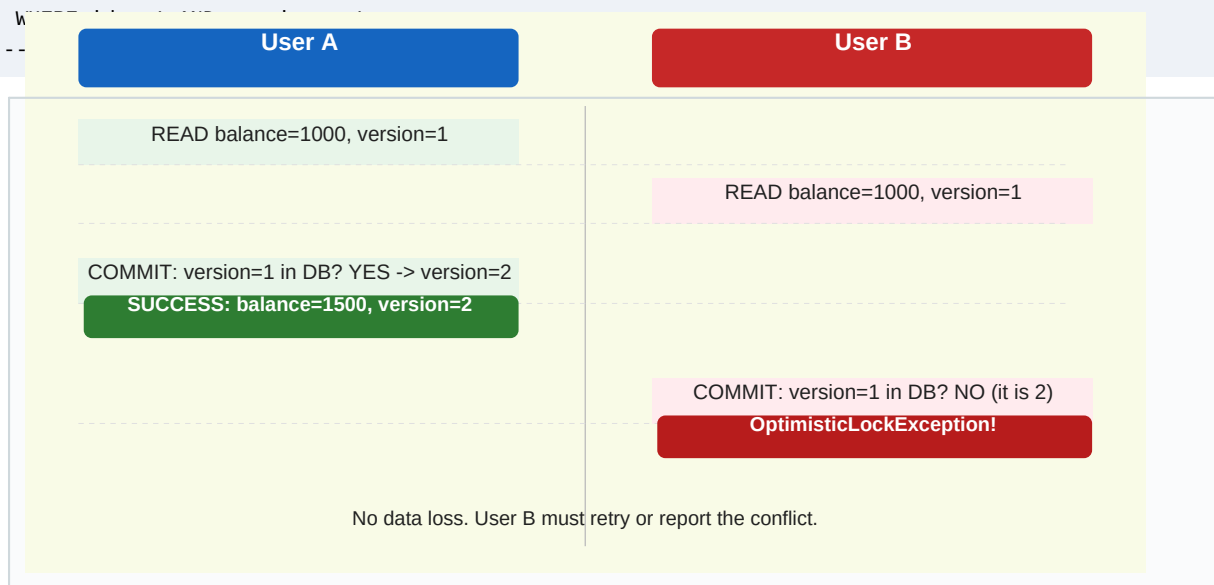


Figure 2 - Optimistic Locking: User A commits first, version advances. User B commit is rejected.

Key Rules

- Add @Version to every entity that concurrent users can edit.
- JPA increments the version automatically; never set it manually.
- The version column works with int, long, short, Integer, Long, and java.sql.Timestamp.
- If the UPDATE affects 0 rows, JPA throws OptimisticLockException (a RuntimeException).
- Best for read-heavy workloads where conflicts are uncommon.

Q1. What annotation enables optimistic locking in JPA?

@Version placed on an int (or Long) field inside the entity class.

Q2. Who increments the version value?

JPA does it automatically on every successful UPDATE. You must never set it yourself.

Q3. What happens when two users commit the same version at the same time?

The first commit succeeds and the version advances. The second commit finds a version mismatch and JPA throws OptimisticLockException, rolling back the second transaction.

Q4. Does optimistic locking block other users while you work?

No. Reads are completely free. Only the final commit checks for conflicts, so other users are never waiting.

Q5. Can I use a timestamp instead of an integer for @Version?

Yes. java.sql.Timestamp is supported, but integers are preferred because clocks on multiple servers can be slightly out of sync.

3 Handling OptimisticLockException

Catch, retry, and keep the user informed

Why You Must Handle the Exception

`OptimisticLockException` is a **RuntimeException** that JPA throws when a version mismatch is detected at commit time. Spring `@Transactional` will roll back the transaction automatically. Your job is to catch the exception (or `ObjectOptimisticLockingFailureException` in Spring Data), reload the entity, re-apply the change, and retry. If retries keep failing you should tell the user that the record was changed by someone else.

Simple retry pattern

```
@Service
public class AccountService {

    @Autowired
    private AccountRepository repo;

    public void depositWithRetry(Long id, int amount) {
        int attempts = 0;
        while (attempts < 3) {
            try {
                Account acc = repo.findById(id).orElseThrow();
                acc.setBalance(acc.getBalance() + amount);
                repo.save(acc); // may throw if version changed
                return;         // success - exit loop
            } catch (ObjectOptimisticLockingFailureException ex) {
                attempts++;
                if (attempts == 3)
                    throw new RuntimeException("Conflict, please retry.", ex);
            }
        }
    }
}
```

Using Spring Retry annotation (cleaner)

```
@Retryable(
    value = ObjectOptimisticLockingFailureException.class,
    maxAttempts = 3,
    backoff = @Backoff(delay = 100) // wait 100 ms between tries
)
@Transactional
public void deposit(Long id, int amount) {
    Account acc = repo.findById(id).orElseThrow();
    acc.setBalance(acc.getBalance() + amount);
    repo.save(acc);
}
```

Key Rules

- `OptimisticLockException` IS-A `RuntimeException`; Spring rolls back automatically.
- Spring Data wraps it as `ObjectOptimisticLockingFailureException`.
- Always start a fresh transaction for each retry - reusing the old one will not help.
- Three retries is a sensible default; after that, surface an error to the user.
- Never swallow the exception silently - the data would still be lost.

Q1. What does OptimisticLockException signal?

That the row was changed by another transaction between the time you read it and the time you tried to commit your update.

Q2. Does catching OptimisticLockException and doing nothing help?

No. The transaction is already rolled back. You must start a new transaction, reload the entity, re-apply the change, and save again.

Q3. What is ObjectOptimisticLockingFailureException?

A Spring Framework wrapper around the JPA OptimisticLockException. It is thrown by Spring Data repositories.

Q4. When should I stop retrying and show an error?

After a small fixed number of attempts (typically 3). Endless retries can hide a deeper problem and degrade performance.

Q5. Can @Retryable be used with @Transactional together?

Yes, but @Retryable must be on the outer method so each retry starts a brand-new transaction.

4 Pessimistic Locking

SELECT FOR UPDATE - lock the row the moment you read it

How Pessimistic Locking Works

Pessimistic locking assumes conflicts are *likely*. As soon as you read a row, the database places a lock on it. Any other transaction that tries to read or update the same row will **block** (wait) until your transaction commits or rolls back. This guarantees no other writer can touch the row while you have it - but it does reduce throughput because other users must queue up.

Using em.find with a lock mode

```
@Transactional
public void transfer(Long fromId, Long toId, int amount) {

    // Lock BOTH rows immediately at read time
    Account from = em.find(Account.class, fromId,
        LockModeType.PESSIMISTIC_WRITE);
    Account to    = em.find(Account.class, toId,
        LockModeType.PESSIMISTIC_WRITE);

    from.setBalance(from.getBalance() - amount);
    to.setBalance(to.getBalance() + amount);
    // Locks released automatically when transaction commits
}
```

Using Spring Data repository with @Lock

```
public interface AccountRepository extends JpaRepository<Account, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT a FROM Account a WHERE a.id = :id")
    Account findById(Long id);
}
```

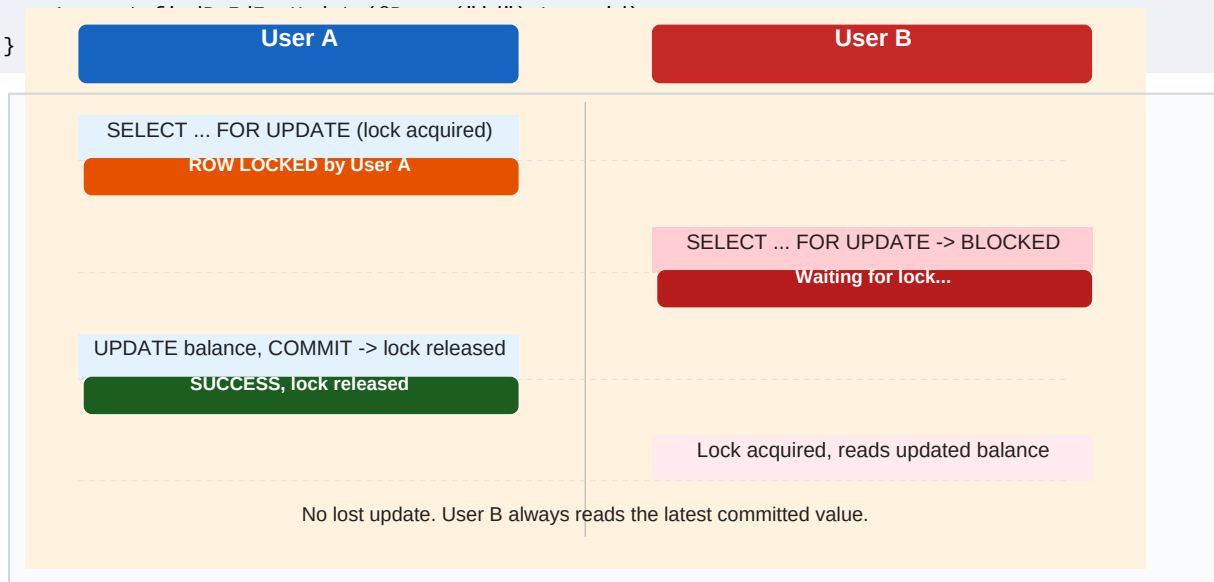


Figure 3 - Pessimistic Locking: User B is blocked until User A lock is released at commit.

Key Rules

- Pessimistic locking prevents conflicts by blocking concurrent writers.
- Use `LockModeType.PESSIMISTIC_WRITE` to block both readers and writers.
- Always keep pessimistic transactions short to avoid long waits.

- Lock rows in a consistent order across transactions to prevent deadlocks.
- Best suited for financial transfers, reservations, and other conflict-heavy operations.

Q1. What SQL does PESSIMISTIC_WRITE generate?

SELECT ... FROM table WHERE id = ? FOR UPDATE - the database locks the row immediately.

Q2. What happens if two transactions try to lock the same row at the same time?

One acquires the lock and proceeds; the other blocks (waits) until the first transaction commits or rolls back, then it acquires the lock.

Q3. What is a deadlock and how do I avoid it?

A deadlock occurs when Transaction A holds a lock that B needs, and B holds a lock that A needs. Always acquire multiple locks in the same order in every transaction.

Q4. How does pessimistic locking differ from optimistic in user experience?

With optimistic locking, the user gets an error at save time and must retry. With pessimistic locking, the request simply waits silently until the row is free.

Q5. What is a lock timeout and why is it important?

A lock timeout limits how long a transaction waits for a lock. Without it, a slow transaction can make other users wait indefinitely.

5 LockModeType Options

Choosing the right lock mode for the job

All LockModeType Values

JPA defines several lock modes through the **LockModeType** enum. You pass one of these to **em.find()**, **em.lock()**, **em.refresh()**, a JPQL query, or the **@Lock** annotation on a Spring Data repository method. Choosing the wrong mode wastes either safety or performance.

LockModeType	What it does	SQL hint	Best for
NONE	No locking at all. Default behaviour.	No extra SQL	Read-only queries
OPTIMISTIC (READ)	Loads version; checks at commit that no other transaction changed the row.	No extra SQL at read; version check at commit	Entities with @Version; rare conflicts
OPTIMISTIC_FORCE_INCREMENT (WRITE)	Same as OPTIMISTIC but always increments version even if you did not modify the entity.	Forced UPDATE to version at commit	When reading counts as a logical modification
PESSIMISTIC_READ	Shared lock. Others can still read but cannot write.	SELECT ... LOCK IN SHARE MODE	Reports that must not see mid-transaction writes
PESSIMISTIC_WRITE	Exclusive lock. Others cannot read or write.	SELECT ... FOR UPDATE	Financial transfers, seat reservations
PESSIMISTIC_FORCE_INCREMENT	Exclusive lock plus version increment. Combines both strategies.	SELECT ... FOR UPDATE	Pessimistic lock with version audit trail

Code examples for each mode

```
// 1. OPTIMISTIC (explicit, though @Version alone is enough)
Account a = em.find(Account.class, 1L, LockModeType.OPTIMISTIC);

// 2. PESSIMISTIC_READ -- shared lock, others can read but not write
Account b = em.find(Account.class, 1L, LockModeType.PESSIMISTIC_READ);

// 3. PESSIMISTIC_WRITE -- exclusive lock
Account c = em.find(Account.class, 1L, LockModeType.PESSIMISTIC_WRITE);

// 4. On a JPQL query
List<Account> locked = em
    .createQuery("SELECT a FROM Account a WHERE a.balance < 0",
        Account.class)
    .setLockMode(LockModeType.PESSIMISTIC_WRITE)
    .getResultList();
```

Key Rules

- PESSIMISTIC_WRITE is the safest choice for concurrent writes.
- PESSIMISTIC_READ allows concurrent reads but blocks concurrent writes.

- OPTIMISTIC requires @Version on the entity; without it the mode is ignored.
- PESSIMISTIC_FORCE_INCREMENT is rarely needed but useful for audit-trail scenarios.
- Always set a lock timeout hint to avoid infinite waits.

Q1. What is the difference between PESSIMISTIC_READ and PESSIMISTIC_WRITE?

PESSIMISTIC_READ acquires a shared lock - others can still read but not write. PESSIMISTIC_WRITE acquires an exclusive lock - others can neither read nor write.

Q2. What does OPTIMISTIC_FORCE_INCREMENT do differently from OPTIMISTIC?

It increments the version column even if you only read the entity and never changed a field. Useful when reading is itself a meaningful business action.

Q3. How do I set a timeout for a pessimistic lock?

Pass a hint map: `em.find(Account.class, id, LockModeType.PESSIMISTIC_WRITE, Map.of("javax.persistence.lock.timeout", 2000))`.

Q4. Which lock mode should I use for a daily report query?

PESSIMISTIC_READ if you need a consistent snapshot during the query, or NONE if slight inconsistencies are acceptable.

Q5. Does LockModeType.NONE provide any guarantee?

No. It is identical to not specifying a lock at all. Use it explicitly only as documentation that locking is intentionally omitted.

Side-by-Side Comparison

Neither strategy is universally better. The right choice depends on how often conflicts occur, how long transactions run, and how your users expect the application to behave when a conflict happens.

Dimension	Optimistic Locking	Pessimistic Locking
Core idea	Let everyone read freely; detect conflicts at commit.	Block competitors the moment you read the row.
When conflict detected	At commit time (end of transaction).	At read time (start of the critical section).
Effect on other users	No waiting. All users read at full speed.	Other users block and wait for the lock.
User experience on conflict	Error at save time: must reload and retry.	Transparent wait, then success (usually).
Database overhead	Only an extra version column; no lock manager.	Lock manager involved; risk of deadlocks.
Best scenario	Read-heavy; conflicts rare (e.g., profile edits).	Write-heavy; conflicts frequent (e.g., bank transfers).
Risk	Late conflict detection; retry logic needed.	Long transactions can cause waits and deadlocks.
JPA setup	@Version field on entity.	LockModeType.PESSIMISTIC_WRITE at em.find().

Decision guide

Conflicts happen less than 5% of the time -> Use Optimistic Locking.

Conflicts happen frequently -> Use Pessimistic Locking.

Transactions are very short (under 1 second) -> Pessimistic is fine.

Transactions are long (seconds to minutes) -> Prefer Optimistic to reduce blocking.

You are building a banking or reservation system -> Pessimistic for correctness.

You are building a social app (likes, profile edits) -> Optimistic for throughput.

You can afford retry logic -> Optimistic; otherwise Pessimistic.

Combining both strategies

It is valid to use both on the same application - even on the same entity. For example, a shopping cart might use optimistic locking for item quantity updates (conflicts rare) but pessimistic locking when finalising a payment (correctness critical).

```
// Optimistic -- good for profile updates (add @Version to entity)
@Version
private int version;    // JPA handles the rest automatically
```

```
// Pessimistic -- good for payment processing
@Transactional
public void processPayment(Long accountId, int amount) {
    Account acc = em.find(Account.class, accountId,
        LockModeType.PESSIMISTIC_WRITE);
    if (acc.getBalance() < amount)
        throw new InsufficientFundsException();
    acc.setBalance(acc.getBalance() - amount);
} // lock released here at commit
```

Key Rules

- Start with Optimistic locking; switch to Pessimistic only if contention is a measured problem.
- Always measure first - do not guess whether conflicts are frequent.
- Optimistic locking scales better in distributed and cloud environments.
- Pessimistic locking is simpler to reason about but is a concurrency bottleneck at scale.
- Both strategies can coexist in the same application on different entities or operations.

Q1. Which strategy scales better under high read traffic?

Optimistic locking. Reads do not acquire any lock, so thousands of users can read concurrently without blocking each other.

Q2. Which strategy guarantees no conflict ever reaches the user?

Pessimistic locking. Conflicts are resolved by blocking before any user makes a change, so no rollback or retry is ever seen by the user.

Q3. Can I use both strategies at the same time?

Yes. Add `@Version` to the entity (enables optimistic locking) and also call `em.find` with `PESSIMISTIC_WRITE` when you need both a lock and a version audit trail.

Q4. What happens if I use optimistic locking in a high-contention scenario?

Many transactions will throw `OptimisticLockException`, causing repeated retries. Throughput drops and the user experience degrades. Switch to pessimistic locking.

Q5. Does Hibernate second-level cache interact with locking?

Yes. Pessimistic locking always bypasses the second-level cache and goes straight to the database. Optimistic locking reads from cache but still validates the version at commit.